ASMSA and ASMSA_v2

Scientific and implementation comparison

Compared: `/home/tedeschg/prj/ASMSA` and `/home/tedeschg/prj/ASMSA_v2`.

Snapshot: 31 October 2026. v2 checked out at `f065c7e`; original checked out at `219fb9e`. The v2 working tree includes a local edit to `notebooks/06b-md-live.ipynb`; that file was not edited for this report.

This edition is more scientific in a specific sense: it describes the objective functions implemented in source code, distinguishes mathematical claims from repository facts, and states important limits on what the algorithms and scores can establish. It is not a controlled performance or scientific-validation study.

Highlight convention

Color	Meaning
Red tint	Original implementation
Green tint	v2 implementation
Citations [n]	Published scientific/software literature listed at the end

Executive scientific summary

Both projects build learned low-dimensional coordinates from molecular trajectory features and connect those coordinates to analysis or PLUMED metadynamics. The v2 rewrite is not a change to the overall scientific question; it changes the software architecture, the available regularizers and diagnostics, and important parts of data handling and simulation export.

Question	Finding from source inspection
What is the learned object?	A deterministic encoder maps a standardized feature vector to a low-dimensional latent vector. It is not, by itself, a validated reaction coordinate or a kinetic model.
What changed in the AAE game?	Original uses a custom randomized linear raw-logit discriminator loss; v2 defaults to BCE-with-logits, spectral normalization, and separate optimizer steps. They are different optimization games, not equivalent implementations.
What stayed similar?	The core bond/angle/dihedral/neighbor-distance feature machinery was substantially carried forward. Exact AST matches were found for 23 named definitions, with additional altered same-name definitions.
What is scientifically new in v2?	Explicit inter-component distances, options for MMD and sliced-Wasserstein prior matching, VAMP-2/kinetic diagnostics, time-aware split tools, multi-molecule adapters, and safer PLUMED mapping/export.
Does the code prove v2 is better?	No. No same-data, same-seed, repeated-run benchmark or independent scientific validation was performed for this report. More options and tests establish capability, not superior scientific results.

Evidence and scope

- Repository claims come from the checked-out source, notebooks, configuration, tests, and current commit metadata. The two repositories are separate checkouts; this is a file/content comparison, not a clean common-ancestor patch review.
- Counts exclude notebooks, tests, caches, and generated files: original Python package 12 files / about 2,310 lines; v2 package 66 files / about 9,583 lines. This measures software surface, not scientific merit.
- Scientific references support general methods. They do not validate this project's trained weights, chosen hyperparameters, simulated trajectories, atom map, or physical conclusions.

Part I - Scientific background and mathematical formulation

1. Molecular dynamics, state variables, and collective variables

Let R_t denote the coordinates of all atoms at simulation time t, and let U(R) be the molecular potential energy. Molecular dynamics generates a time-correlated sequence of configurations R_0, R_1, ... A collective variable (CV) is a low-dimensional map s(R) = (s_1(R), ..., s_d(R)) used to summarize a process such as binding, opening, or conformational change. The purpose is to retain coordinates relevant to the slow process while discarding much of the atom-level detail.

This creates a scientific distinction: reconstructing structure from s is evidence that s retains information about the input features; it does not by itself show that s is a good reaction coordinate, that barriers are absent in the omitted coordinates, or that transition kinetics can be recovered. The v2 project adds kinetic diagnostics, but the CV still requires physical interpretation and independent validation [6, 7].

2. Feature map and scaling

At time t, R_t means the coordinates of all atoms. The feature-calculation step turns those coordinates into x_t: a vector of selected molecular measurements, such as bond lengths, angles, dihedrals, and selected atom-pair distances. Here x_t simply means the feature vector; it is not a phi-angle variable.

Per-feature affine scaling used by the package

Standardized feature j = (feature j - training mean j) / training scale j

With standard scaling, mu_j and sigma_j are estimated from training frames only. This matters because a wide-valued feature block can dominate an unweighted squared reconstruction error. Scaling makes feature magnitudes more comparable, but it does not make all scientific features equally important, remove correlations, cure outliers, or guarantee that the encoder uses a feature meaningfully. v2 additionally offers robust and pass-through modes; standard mean/std is the compatibility choice.

3. Autoencoder and aggregate posterior

Write E_theta for the encoder and G_psi for the decoder. In this repository the learned mapping is deterministic at inference: z_t = E_theta(x_t), with reconstructed features xhat_t = G_psi(z_t). The basic reconstruction objective is an average elementwise loss; for the default squared loss it is

Reconstruction objective

```
L_rec(theta, psi) = E_{x ~ p_data} [ || G_psi(E_theta(x)) - x ||_2^2 ]
```

The distribution of encoded points over the data is the aggregate posterior q_E(z) = integral delta(z - E_theta(x)) p_data(x) dx. The delta notation here means that each input maps to its encoder output. In an AAE, a discriminator is trained to distinguish q_E samples from a selected prior p(z), while the encoder is trained to make q_E resemble p. This is distribution matching over a batch/population; it does not force the code of each frame to equal a particular prior sample [1].

Idealized distribution-matching statement

```
q_E(z) approximately equals p(z)
```

A prior can define the latent geometry (for example a normal prior or a ring-like Gaussian mixture). Matching the aggregate distribution does not preserve temporal ordering: two encoded trajectories with identical point clouds but different time order have the same aggregate distribution. Consequently, prior matching alone does not certify slow-mode quality.

4. Critical implementation difference: original raw-logit game versus v2 BCE game

The source code matters here. The original comments that BCE-with-logits is hard-coded, but its actual loss multiplies raw discriminator outputs by random factors between 1.0 and 1.05. If D(z) is the scalar raw output, z_E is an encoded sample and z_P is a prior sample, the implemented discriminator minimizes approximately

Original ASMSA: discriminator and encoder updates (schematic from code)

```
L_D^old = E[a D(z_E)] - E[b D(z_P)]    where a,b ~ Uniform(1, 1.05)
L_E^old,adv = - E[c D(E_theta(x))]      where c ~ Uniform(1, 1.05)
```

The signs encourage the discriminator to assign larger outputs to prior samples than to encoder samples, and the encoder to raise its own outputs. Random multipliers perturb the update. This is a linear raw-logit game. It is not the standard binary cross-entropy GAN objective, and the source shown does not add a gradient penalty or spectral normalization in this old training step. It should not be retroactively described as BCE or as a Wasserstein GAN.

v2's default adversarial option instead uses BCE-with-logits targets (prior=real, encoder=fake) and a non-saturating encoder loss. Its discriminator linear layers use spectral normalization by default; optional R1 regularization penalizes discriminator input-gradient magnitude at prior samples. Separate optimizer instances and learning rates are used for autoencoder, discriminator, and, for the alternating mode, encoder generator update.

ASMSA_v2: default adversarial objective (schematic)

```
L_D^new = BCEWithLogits(D(z_P), 1) + BCEWithLogits(D(z_E), 0)
L_E^new = L_rec + lambda * BCEWithLogits(D(E_theta(x)), 1)
Optional: L_D^new += (gamma_R1 / 2) E[ || grad_(z_P) D(z_P) ||_2^2 ]
```

The defaults are not interchangeable. The two games have different gradient scales and equilibrium behavior; reusing learning rates or claiming identical training because both contain an encoder and discriminator is not justified. If v2 uses `adv_mode="joint"`, reconstruction and the encoder adversarial term share one backward pass. The alternating option separates them into optimizer steps. The code documentation notes the single-encoder-forward design for the joint route; alternating performs another encoder forward by design.

5. v2 non-adversarial regularizers

v2 offers two additional ways to encourage q_E to match p. These are alternatives to the adversarial game, not additional terms that are always active.

MMD: empirical two-sample penalty (kernel k)

```
MMD^2(Q,P) = E[k(z,z')] + E[k(p,p')] - 2 E[k(z,p)]
k_IMQ(u,v) = C / (C + ||u-v||_2^2)
```

The implementation uses an inverse-multiquadratic kernel over several bandwidth scales and an unbiased within-sample U-statistic (diagonal terms omitted). An unbiased finite-sample estimate can be slightly negative. MMD is a kernel two-sample discrepancy [2, 3]; the estimate depends on batch size, kernel, and bandwidth scales.

Sliced-Wasserstein: projected one-dimensional sample matching

```
SW_p^p(Q,P) approx (1/L) sum_l (1/n) sum_i | theta_l^T z_(i) - theta_l^T p_(i) |^p
where theta_l are random unit directions; projected samples are sorted.
```

The code defaults to 64 random projections and p=2; if sample counts differ, sorted values are resampled on a common quantile grid. It optimizes the average powered projected discrepancy (a Monte Carlo sliced-Wasserstein penalty), not an exact full-dimensional optimal-transport solve. Finite random projections add estimator variation [4].

6. Feature choice and multi-molecule training

For a complex, within-component features describe each partner's shape; cross-component distances encode relative arrangement. v2's `ComplexMolecule` constructs inter-component distances densely or samples a seeded sparse subset. `auto_features` offers CA-by-heavy-atom, contact-based, all-pairs, and related selection choices. These choices change the information and dimensionality given to the model. A sparse subset can miss a critical contact; a dense set can create many correlated features. They are modeling choices to validate, not neutral preprocessing.

v2's input/output adapters map molecule-specific feature widths to/from a shared neural-network trunk. Batches contain one molecule at a time; the latent discriminator sees z, not the molecule identifier. This architecture encourages comparable latents across systems, but does not prove that the shared coordinates correspond to the same physical mechanism in every system. Demonstrate that with held-out behavior and structural interpretation.

Part II - Data protocol, model selection, and enhanced sampling

7. Train/validation/test splitting: a real methodological change

The original `02-prepare.ipynb` first shuffles the trajectory configurations, then slices the shuffled frames into 70% train, 15% validation, and 15% test. It subsequently computes cross-split RMSD matrices and filters similar structures from train and test using a 0.05 nm cutoff. Thus, it is inaccurate to say that the original used an unfiltered random split: it used shuffled coverage plus an explicit RMSD-based de-duplication procedure.

v2's default is a chronological 70/15/15 split. This confines direct frame-adjacency leakage mainly to split boundaries, rather than interleaving nearby frames throughout all sets. v2 also ports RMSD-based train filtering as an optional step. Its source comments explicitly warn that random/strided splits need decorrelation to reduce leakage.

Protocol	Scientific advantage	Important limitation
Old: shuffle, slice, RMSD cross-check/filter	Each split samples the full observed time range; RMSD filtering removes many geometrically close cross-split structures.	Time correlations are not fully removed by one geometric threshold; shuffling can conceal drift and does not create independent samples.
v2 default: chronological blocks	Easier to test forward-in-time transfer; neighboring frames mostly remain within one split.	A later block may have a different state/population because of drift or incomplete equilibration; boundary correlation remains.
v2 optional RMSD filter	Can additionally remove training frames close to held-out structures.	RMSD is not a universal measure of dynamical independence; results depend on alignment atoms/cutoff and may remove useful states.

Neither protocol makes frames independent in the statistical sense. The correct choice depends on the claim: random coverage with de-duplication can ask about interpolation among sampled states; chronological holdout can ask about prediction on a later segment. Report the split rule, time range, alignment atoms, cutoff, and state populations. Never tune on the final test set.

8. Training and validation implementation

In original code, training is embedded in Keras `AAEModel.train_step`; the autoencoder update, discriminator update, and encoder "cheat" update occur sequentially. The same compiled Keras optimizer object is used. It also recomputes encoder outputs for later losses, so batch-normalization running statistics can be updated repeatedly. The old notebook uses validation data and early stopping in the documented training workflow, but the tuner's missing-validation fallback takes a strided subset of the training array rather than an independent holdout.

v2 separates model definition from `Trainer`, uses separate optimizers, and has an explicit evaluation pass and best-weight restoration. It supports non-shuffled, non-dropped validation loaders. These software controls make evaluation easier to reproduce; they do not independently guarantee an unbiased estimate if the user reuses validation repeatedly for hyperparameter selection.

9. KL prior-match score: what the tuning number means

Both codebases use the k-nearest-neighbor estimator attributed in the source to Pérez-Cruz [5]. In this project's convention x is sampled from the prior P and y from encoded points Q, so the estimated direction is D_KL(P || Q), not D_KL(Q || P). The score is a tuning metric; it is not the direct training loss.

kNN KL estimate used by both generations (notation consistent with code)

```
D_KL(P || Q) approx -(d/n) sum_i log(r_i / s_i) + log(m / (n-1))
r_i = 2nd-neighbor distance of x_i among P; s_i = nearest y distance among Q.
```

The original tuner clips negative estimates to zero and aggregates the late-epoch mean plus variance over the selected tail. v2 retains this objective for comparability; it also caps the evaluated frame count in its KL tracker. kNN density-ratio estimates can be noisy in finite samples, especially as dimension grows; the direction is asymmetric and the estimate depends on sample size and support coverage. A low score does not certify a useful CV or good transitions.

10. VAMP-2: a temporal score, not a reconstruction score

v2 adds VAMP-style evaluation from time-ordered latent sequences. Given z_t and z_(t+tau), the implementation forms centered lagged matrices X and Y, their covariance estimates C00, C11, C01, and the whitened matrix K = C00^(-1/2) C01 C11^(-1/2). It then computes singular values sigma_i and returns

Implemented VAMP-2 convention

```
S_VAMP2 = 1 + sum_i clip(sigma_i, 0, 1)^2
```

The added one represents the trivial stationary component under the package's mean-free convention. The code also computes implied timescales t_i(tau) = -tau*dt / log(sigma_i), with clipping for numerical stability. VAMP theory motivates variational scores for slow dynamical processes [6, 7]. However, this implementation's score depends on the lag, trajectory length/order, stationarity, covariance conditioning, and finite sampling. Its usefulness requires a time-ordered and physically appropriate validation trajectory; shuffled data makes the score meaningless.

A VAMP-2 score says that the represented functions retain time-lagged predictive structure under the chosen estimator. It does not by itself show that metadynamics along those coordinates will produce correct kinetics, eliminate hidden barriers, or converge a free-energy surface.

11. Composite objective: a heuristic, not a dimensionless law

The v2 code combines the prior-match KL score, reconstruction loss, and normalized VAMP score as `KL + reconstruction - (VAMP2 - 1)/latent_dim`. These terms have different numerical scales and meanings. The code does not normalize them by uncertainty or assign calibrated weights. Therefore "composite" is an engineering ranking heuristic; a change in feature scaling, reconstruction reduction, or score magnitude can change which term dominates. Always inspect the components separately and test sensitivity to the objective.

12. PLUMED and metadynamics

Metadynamics adds history-dependent Gaussian hills in CV space. A simplified bias is

Metadynamics bias

```
V(s,t) = sum_(k: t_k < t) w_k exp( -1/2 sum_i [(s_i - s_i(k))/sigma_i]^2 )
```

In well-tempered metadynamics (WTMetaD), hill heights decrease as bias accumulates. With bias factor gamma = (T + DeltaT)/T, the asymptotic bias is proportional to -(1 - 1/gamma) F(s), up to a constant and conventions. PLUMED describes Gaussian hill deposition, HILLS/grid storage, and bias/reweighting behavior [8, 9, 10]. The v2 exporter defaults to a bias factor of 15, deposition pace of 1000 steps, height 1.0, and CALC_RCT for the time-dependent reweighting offset. These are workflow defaults, not universally optimal parameters.

The code chooses sigma_i = observed CV range / 20 and expands each grid side by three observed ranges (the selected reference is the test subset when it contains enough frames in the notebook). This is a practical heuristic, not a statistical confidence interval. A test set that misses a basin can understate the needed range; too-small sigma can lead to inefficient bias, and a too-large sigma can blur basins. Examine pilot runs and convergence rather than treating this default as a physical constant.

13. Export correctness: features, topology, and periodic boundaries

For the exported CV to be the same function learned in training, the deployed function must compose all operations in the correct order: atom coordinates -> feature map -> saved scaling -> encoder. v2 packages that composition in TorchScript and offers a Python batch model plus a PLUMED-compatible single-frame model. It explicitly traces in evaluation mode and carries the scaler inside the model. This addresses an important class of deployment mismatch.

The old PLUMED notebook derives an atom ordering from a GROMACS index file and says to check it manually. v2 maps atom identity using chain, residue number/name, and atom name (with known residue/atom aliases), reports unmatched atoms, and refuses PLUMED generation in strict mode. The mapping is more auditable but can still fail if PDB and topology disagree or keys are ambiguous; successful mapping is not proof that the files describe the same physical state.

The current v2 commit explicitly records a CV mismatch caused by PLUMED POSITION actions wrapping atoms independently. Its exporter emits POSITION actions with NOPBC and WHOLEMOLECULES support. This is a concrete correctness fix: in a periodic box, a molecule may be split across boundaries, so coordinate differences can represent box wrapping instead of physical geometry. The correct atom grouping and PBC treatment must match the featurizer's coordinate convention. Run the provided bias-free `plumed driver` recomputation and compare Python and PLUMED CVs on identical frames before production.

Part III - Source-level differences and scientific interpretation

14. Old-to-new implementation map

Original source / operation	v2 implementation	Nature of change
`aae_model.py`: model construction and custom train_step	`models/aae.py`, encoder/decoder/discriminator/adapters; `train/trainer.py`	Framework rewrite and changed adversarial loss/optimizer protocol.
`utils.py`: priors, sparse constraints, helper losses	`priors/`, `losses/`, `utils/config.py`	Public typed components; new MMD/SW penalties; several old options not one-for-one.
`molecule.py`, `mol_model.py`, `helpers.py`	`featurize/`	Core geometry largely retained; inter-component distances and feature selection added.
Notebook split/normalization and Keras data pipeline	`data/split.py`, dataset.py, normalize.py, featurize_frames.py	Explicit reusable split, scaler, multi-system batch, and lagged-pair tools.
`aae_hyper_model.py`, `tunewrapper.py`, analyzer	`tune/`, `metrics/`	Keras Tuner replaced; old KL objective retained, new temporal/reconstruction objectives added.
Notebook PLUMED adapter and index-file atom ordering	`export/torchscript.py`, `export/plumed.py`	Packaged scaler+feature+encoder, identity mapping, strict checks, NOPBC and PLUMED input writer.
`gmx.py` and interactive notebook commands	`md/gromacs.py`, runner.py, protocol.py, mdp.py, analysis.py	Execution backends, resumable stages, and analysis separated into APIs.
Training plots/visualizer and standalone analysis notebooks	`viz/`, `analysis/`, `metrics/`	More analysis is reusable package code; notebook analysis still remains part of workflow.

15. Code excerpts: changed objective and export

The old objective is not the v2 objective

ORIGINAL ASMSA

package/asmsa/aae_model.py (train_step)

```
# Original: raw discriminator outputs
neg = D(z_encoded) * U(1, 1.05)
pos = -D(z_prior) * U(1, 1.05)
loss_D = mean(neg + pos)
loss_encoder = -mean(D(E(x)) * U(1, 1.05))
```

ASMSA_v2

src/asmsa_v2/losses/adversarial.py and train/trainer.py

```
# v2: BCE-with-logits targets
loss_D = BCE(D(z_prior), 1) + BCE(D(z_encoded), 0)
loss_encoder = loss_recon + lambda * BCE(D(E(x)), 1)
# spectral norm default; optional R1 penalty
```

The excerpts are abbreviated to show mathematical behavior; random multipliers and exact tensor reductions are omitted from the green equations where not needed. The report does not imply equality of optimizers, initializations, or trained representations.

Notebook-derived atom order versus identity-resolved map

ORIGINAL ASMSA

original 05-plumed.ipynb

```
# Original notebook: sort by index-file order
pdb2gmx = nhs[np.argsort(ndx)] + 1
# comment asks user to double-check manually
```

ASMSA_v2

src/asmsa_v2/export/plumed.py

```
# v2: resolve atom identities or fail in strict mode
atom_map = map_pdb_to_topology(
    pdb, topology, featurized_atoms=used_atoms,
    strict=True,
)
# generated POSITION uses NOPBC
```

16. What remained similar

An AST-level comparison found 23 exact same-named definitions across old and v2 package paths, including bond/dihedral feature-map operations, neighbor-distance classes, and portions of topology parsing and molecular feature assembly. Ten other same-named definitions had changed syntax trees, including molecule/model construction and angle/internal-coordinate/helper pieces. This supports a precise summary: the feature engine is partly preserved and partly modified; the system around it has been extensively redesigned.

17. What can and cannot be concluded scientifically

Supported by the inspected code	Not established by this comparison
v2 exposes more explicit choices for loss, splitting, scaling, analysis, and MD execution.	That v2 has lower error, converges faster, produces better CVs, or yields more accurate free energies.
A separate PLUMED atom identity resolver and Python-vs-PLUMED verification path exist.	That every real protein/ligand topology maps correctly or that a given production trajectory is PBC-correct.
VAMP-2 and implied-timescale routines are implemented for ordered latent trajectories.	That learned latents span the true reaction coordinate or recover physical kinetics in any particular system.
The old and new adversarial losses are mathematically different; MMD/SW are additional alternatives.	That one regularizer is generally superior; performance depends on data, prior, latent dimension, scaling, and optimization.
The notebooks provide a reproducible project workflow and the repository has a test suite.	That repository tests constitute validation of a scientific hypothesis or certify the methods on all systems.

18. Minimum scientific validation plan for a fair comparison

A defensible old-versus-new empirical comparison should freeze the scientific inputs, vary only the implementation, and report repeated runs. At minimum:

- Use identical trajectories, topology, atom selections, featurization, split indices, and train-only scaling. Document any unavoidable differences in feature ordering or units.
- Compare the old objective to v2's adversarial objective first; compare MMD/SW as separate ablations. Do not mix loss change with framework change in a single "version" result.
- Use matched latent dimension, architecture capacity as closely as possible, optimizer budget, learning-rate search budget, batch size, and stopping rule. Report parameter counts because the old masked multi-model design and v2 adapters are not capacity-equivalent.
- Run multiple random seeds and report mean, spread, and failures for reconstruction, prior-match, latent collapse/usable spread, VAMP-2 at multiple lags, and independent test-set metrics.
- For molecular interpretation, compare state populations, transitions, structural endpoints, and known observables. Analyze uncertainty and state coverage; a two-dimensional plot is not evidence of a reaction mechanism.
- Before biased MD, verify CV equality between Python and PLUMED on identical frames, then perform short pilot simulations and monitor grid coverage, reweighted free-energy convergence, and independent replicas.
- Keep the final test trajectory untouched during tuning. Prefer independent trajectories/replicas or temporally separated test blocks when making claims about transfer.

Until such a study is run, the scientifically careful conclusion is: v2 provides a broader and more explicit methodological toolkit and fixes a documented export/PBC mismatch; its superiority in scientific outcomes remains an empirical question.

References

Primary method references. Repository-specific statements cite source paths and commit evidence in the report, rather than treating the papers below as proof about this implementation.

[1] Makhzani, A. et al. "Adversarial Autoencoders." arXiv:1511.05644 (2015; later ICLR workshop version). https://arxiv.org/abs/1511.05644

[2] Tolstikhin, I. et al. "Wasserstein Auto-Encoders." ICLR (2018), arXiv:1711.01558. https://arxiv.org/abs/1711.01558

[3] Gretton, A. et al. “A Kernel Two-Sample Test.” Journal of Machine Learning Research 13 (2012), 723-773. https://jmlr.org/papers/v13/gretton12a.html

[4] Kolouri, S. et al. “Sliced-Wasserstein Autoencoder: An Embarrassingly Simple Generative Model.” arXiv:1804.01947 (2018). https://arxiv.org/abs/1804.01947

[5] Pérez-Cruz, F. “Kullback-Leibler divergence estimation of continuous distributions.” IEEE International Symposium on Information Theory (2008). DOI 10.1109/ISIT.2008.4595271. https://doi.org/10.1109/ISIT.2008.4595271

[6] Mardt, A. et al. “VAMPnets for deep learning of molecular kinetics.” Nature Communications 9, 5 (2018). DOI 10.1038/s41467-017-02388-1. https://doi.org/10.1038/s41467-017-02388-1

[7] Noé, F. and Nüske, F. “A Variational Approach to Modeling Slow Processes in Stochastic Dynamical Systems.” Multiscale Modeling & Simulation 11(2) (2013), 635-655. DOI 10.1137/110858616. https://doi.org/10.1137/110858616

[8] Laio, A. and Parrinello, M. “Escaping free-energy minima.” Proceedings of the National Academy of Sciences 99(20) (2002), 12562-12566. DOI 10.1073/pnas.202427399. https://doi.org/10.1073/pnas.202427399

[9] Barducci, A., Bussi, G. and Parrinello, M. “Well-Tempered Metadynamics: A Smoothly Converging and Tunable Free-Energy Method.” Physical Review Letters 100, 020603 (2008). DOI 10.1103/PhysRevLett.100.020603. https://doi.org/10.1103/PhysRevLett.100.020603

[10] Tribello, G. A. et al. “PLUMED 2: New feathers for an old bird.” Computer Physics Communications 185 (2014), 604-613. DOI 10.1016/j.cpc.2013.09.018. https://doi.org/10.1016/j.cpc.2013.09.018

[11] McGibbon, R. T. et al. “MDTraj: A Modern Open Library for the Analysis of Molecular Dynamics Trajectories.” Biophysical Journal 109(8) (2015), 1528-1532. DOI 10.1016/j.bpj.2015.08.015. https://doi.org/10.1016/j.bpj.2015.08.015

[12] Paszke, A. et al. “PyTorch: An Imperative Style, High-Performance Deep Learning Library.” NeurIPS 32 (2019). https://papers.nips.cc/paper_files/paper/2019/hash/bdbca288fee7f92f2bfa9f7012727740-Abstract.html

[13] Akiba, T. et al. “Optuna: A Next-generation Hyperparameter Optimization Framework.” KDD (2019). DOI 10.1145/3292500.3330701. https://doi.org/10.1145/3292500.3330701

Repository evidence

• Original implementation: `package/asmsa/aae_model.py`, `aae_hyper_model.py`, `molecule.py`, `mol_model.py`, and root notebook `02-prepare.ipynb`.

• v2 implementation: `src/asmsa_v2/train/trainer.py`, `losses/`, `metrics/`, `data/`, `export/`, `featurize/`, and notebooks 02/05/06.

• v2 commit `f065c7e` (31 Oct 2026) records the NOPBC correction for independently wrapped PLUMED POSITION actions and the live-MD notebook addition.

Appendix A - complete package definition inventory

All Python modules in the two installable package source trees are listed below. For each file this index records classes, direct class methods, module-level functions, and locally nested helper functions. It is a navigation aid, not a substitute for reading implementation or tests.

Appendix A - Original ASMSA

package/asmsa/__init__.py
(no local definitions)

package/asmsa/aae_hyper_model.py
def _KLdivergence (line 19)
class _LogCallback -> __init__, on_train_begin, on_train_end, on_epoch_end, _log, get_metric (line 66)
def _LogCallback.__init__ (line 67)
def _LogCallback.on_train_begin (line 77)
def _LogCallback.on_train_end (line 91)
def _LogCallback.on_epoch_end (line 100)
def _LogCallback._log (line 137)
def _LogCallback.get_metric (line 151)
class AAEHyperModel -> __init__, build, fit (line 159)
def AAEHyperModel.__init__ (line 161)
def AAEHyperModel.build (line 190)
def AAEHyperModel.fit (line 202)

package/asmsa/aae_model.py
class AAEModel -> __init__, compile, train_step, test_step, summary, call, call_enc, call_disc (line 15)
def AAEModel.__init__ (line 16)
def AAEModel.compile (line 117)
def AAEModel.train_step (line 139)
def AAEModel.test_step (line 218)
def AAEModel.summary (line 269)
def AAEModel.call (line 274)
def AAEModel.call_enc (line 278)
def AAEModel.call_disc (line 282)
class GaussianMixture -> __init__, sample, prob (line 286)
def GaussianMixture.__init__ (line 287)
def GaussianMixture.sample (line 294)
def GaussianMixture.prob (line 303)

package/asmsa/gmx.py
class GMX -> __init__, start, status, delete, log (line 8)
def GMX.__init__ (line 9)
def GMX.start (line 19)
def GMX.status (line 91)
def GMX.delete (line 96)
def GMX.log (line 102)

package/asmsa/helpers.py
def extract_dihedral_indices_local (line 5)
def extract_atom_indices (line 121)
def extract_atom_indices.parse_pdb_for_atoms (line 140)
def create_gaussian_mixture_on_circle (line 175)

package/asmsa/mol_model.py
def _norm_axis_1 (line 19)
class BondsModel -> __init__, forward (line 27)
def BondsModel.__init__ (line 49)
def BondsModel.forward (line 54)
class AnglesModel -> __init__, forward (line 62)
def AnglesModel.__init__ (line 87)
def AnglesModel.forward (line 98)
class DihedralModel -> __init__, forward (line 115)
def DihedralModel.__init__ (line 137)
def DihedralModel.forward (line 142)
class NBDistancesSparse -> __init__ (line 174)
def NBDistancesSparse.__init__ (line 190)
class NBDistancesDense -> __init__ (line 219)
def NBDistancesDense.__init__ (line 232)
class MoleculeModel -> __init__, forward, get_indices (line 250)
def MoleculeModel.__init__ (line 282)
def MoleculeModel.forward (line 302)
def MoleculeModel.get_indices (line 327)

package/asmsa/molecule.py
def _parse_top (line 14)
def _parse_ff (line 138)
def _match_type (line 197)
class Molecule -> __init__, _match_bonds, _match_angles, _match_dihed, describe_features, get_model,
intcoord (line 220)
def Molecule.__init__ (line 252)
def Molecule._match_bonds (line 310)
def Molecule._match_angles (line 343)

def Molecule._match_dihed (line 382)
def Molecule.describe_features (line 443)
def Molecule.get_model (line 458)
def Molecule.intcoord (line 470)

package/asmsa/plot_training.py
class LiveTrainingPlot -> __init__, on_train_begin, on_epoch_end, _update_plot (line 6)
def LiveTrainingPlot.__init__ (line 12)
def LiveTrainingPlot.on_train_begin (line 36)
def LiveTrainingPlot.on_epoch_end (line 47)
def LiveTrainingPlot._update_plot (line 73)

package/asmsa/tunewrapper.py
class TuneWrapper -> __init__, master_start, master_status, workers_start, workers_status (line 9)
def TuneWrapper.__init__ (line 10)
def TuneWrapper.master_start (line 30)
def TuneWrapper.master_status (line 39)
def TuneWrapper.workers_start (line 46)
def TuneWrapper.workers_status (line 110)

package/asmsa/tuning_analyzer.py
class TuningAnalyzer -> __init__, populate_TB, visualize_tuning, get_best_hp, _analyze (line 12)
def TuningAnalyzer.__init__ (line 13)
def TuningAnalyzer.populate_TB (line 19)
def TuningAnalyzer.visualize_tuning (line 47)
def TuningAnalyzer.get_best_hp (line 123)
def TuningAnalyzer.get_best_hp._get_beautiful_output (line 124)
def TuningAnalyzer._analyze (line 144)

package/asmsa/utils.py
def _compute_number_of_neurons (line 8)
class _Prior -> __init__, __call__ (line 28)
def _Prior.__init__ (line 29)
def _Prior.__call__ (line 32)
class _PriorDistribution -> __init__, __call__ (line 35)
def _PriorDistribution.__init__ (line 36)
def _PriorDistribution.__call__ (line 41)
class _PriorMultivariate -> __init__, __call__ (line 51)
def _PriorMultivariate.__init__ (line 52)
def _PriorMultivariate.__call__ (line 57)
class _PriorImage -> __init__, __call__ (line 60)
def _PriorImage.__init__ (line 61)
def _PriorImage.__call__ (line 74)
class _SparseConstraint -> __init__, __call__ (line 88)
def _SparseConstraint.__init__ (line 89)
def _SparseConstraint.__call__ (line 99)
class _SparseInitializer -> __init__, __call__ (line 102)
def _SparseInitializer.__init__ (line 103)
def _SparseInitializer.__call__ (line 111)
def _masks (line 119)

package/asmsa/visualizer.py
class Visualizer -> __init__, make_visualization (line 8)
def Visualizer.__init__ (line 9)
def Visualizer.make_visualization (line 16)
class VisualizeCallback -> __init__, on_epoch_end (line 58)
def VisualizeCallback.__init__ (line 59)
def VisualizeCallback.on_epoch_end (line 70)

Appendix A - ASMSA_v2

`src/asmsa_v2/__init__.py`
(no local definitions)

`src/asmsa_v2/analysis/__init__.py`
(no local definitions)

`src/asmsa_v2/analysis/clustering.py`
`def _to_np` (line 32)
`class ClusterResult -> summary` (line 39)
`def ClusterResult.summary` (line 47)
`def cluster_latent` (line 61)

`src/asmsa_v2/analysis/export.py`
`def save_cluster_representatives` (line 9)

`src/asmsa_v2/data/__init__.py`
(no local definitions)

`src/asmsa_v2/data/dataset.py`
`def _as_2d` (line 20)
`class MultiMolDataset -> __init__, __len__, __getitem__, tensor, tensors, n_frames, indices_for, round_robin_sampler, loader, eval_loader, __repr__` (line 30)
`def MultiMolDataset.__init__` (line 42)
`def MultiMolDataset.__len__` (line 77)
`def MultiMolDataset.__getitem__` (line 80)
`def MultiMolDataset.tensor` (line 87)
`def MultiMolDataset.tensors` (line 91)
`def MultiMolDataset.n_frames` (line 95)
`def MultiMolDataset.indices_for` (line 98)
`def MultiMolDataset.round_robin_sampler` (line 103)
`def MultiMolDataset.loader` (line 118)
`def MultiMolDataset.eval_loader` (line 144)
`def MultiMolDataset.__repr__` (line 156)
`class RoundRobinBatchSampler -> __init__, _count_batches, _mol_batches, __iter__, __len__` (line 162)
`def RoundRobinBatchSampler.__init__` (line 173)
`def RoundRobinBatchSampler._count_batches` (line 205)
`def RoundRobinBatchSampler._mol_batches` (line 211)
`def RoundRobinBatchSampler.__iter__` (line 224)
`def RoundRobinBatchSampler.__len__` (line 232)
`def multi_mol_collate` (line 237)
`def make_time_lagged_dataset` (line 256)

`src/asmsa_v2/data/featurize_frames.py`
`def featurize_frames` (line 18)

`src/asmsa_v2/data/normalize.py`
`def _as_np` (line 29)
`class FeatureScaler -> __init__, fit, _check_fitted, transform, inverse_transform, fit_transform, save, load, n_features, as_module, report, __repr__` (line 35)
`def FeatureScaler.__init__` (line 52)
`def FeatureScaler.fit` (line 64)
`def FeatureScaler._check_fitted` (line 91)
`def FeatureScaler.transform` (line 96)
`def FeatureScaler.inverse_transform` (line 106)
`def FeatureScaler.fit_transform` (line 116)
`def FeatureScaler.save` (line 120)
`def FeatureScaler.load` (line 136)
`def FeatureScaler.n_features` (line 146)
`def FeatureScaler.as_module` (line 150)
`def FeatureScaler.report` (line 155)
`def FeatureScaler.__repr__` (line 182)
`class StandardizeModule -> __init__, forward` (line 187)
`def StandardizeModule.__init__` (line 194)
`def StandardizeModule.forward` (line 199)

`src/asmsa_v2/data/split.py`
`class SplitIndices -> __post_init__, sizes, summary, _repr_html_, __repr__` (line 35)
`def SplitIndices.__post_init__` (line 43)
`def SplitIndices.sizes` (line 49)
`def SplitIndices.summary` (line 52)
`def SplitIndices._repr_html_` (line 59)
`def SplitIndices.__repr__` (line 73)
`def split_frames` (line 77)
`def decorrelate_by_rmsd` (line 138)
`def lag_pairs` (line 186)

src/asmsa_v2/demo.py
class DemoSystem -> summary, __repr__ (line 41)
def DemoSystem.summary (line 55)
def DemoSystem.__repr__ (line 67)
def _build_topology (line 71)
def _write_mock_gro (line 89)
def make_demo_complex (line 106)
def _rot_x (line 239)
def _rot_y (line 244)
def _rot_z (line 249)

src/asmsa_v2/export/__init__.py
(no local definitions)

src/asmsa_v2/export/plumed.py
def _canon_res (line 68)
def _canon_atom (line 74)
def _atom_key (line 80)
def _format_floats (line 87)
def _position_inputs (line 92)
class PlumedAtomMap -> ok, atoms_string, report, _repr_html_, __repr__ (line 107)
def PlumedAtomMap.ok (line 132)
def PlumedAtomMap.atoms_string (line 136)
def PlumedAtomMap.report (line 140)
def PlumedAtomMap._repr_html_ (line 164)
def PlumedAtomMap.__repr__ (line 183)
def map_pdb_to_topology (line 187)
def map_pdb_to_topology.build_index (line 230)
class CVGrid -> grid_min_str, grid_max_str, sigma_str, report, __repr__ (line 290)
def CVGrid.grid_min_str (line 308)
def CVGrid.grid_max_str (line 312)
def CVGrid.sigma_str (line 316)
def CVGrid.report (line 319)
def CVGrid.__repr__ (line 333)
def cv_grid (line 337)
def write_plumed_dat (line 387)
def write_plumed_driver_dat (line 514)

src/asmsa_v2/export/torchscript.py
class SingleMolEncoder -> __init__, forward (line 45)
def SingleMolEncoder.__init__ (line 52)
def SingleMolEncoder.forward (line 62)
class SingleMolDecoder -> __init__, forward (line 66)
def SingleMolDecoder.__init__ (line 69)
def SingleMolDecoder.forward (line 79)
class BatchCVModel -> __init__, forward (line 83)
def BatchCVModel.__init__ (line 86)
def BatchCVModel.forward (line 97)
class PlumedCVModel -> __init__, forward (line 104)
def PlumedCVModel.__init__ (line 112)
def PlumedCVModel.forward (line 123)
def _freeze_for_export (line 131)
def build_cv_models (line 146)
def _trace_and_freeze (line 175)
def export_cv_models (line 202)
def verify_cv_models (line 291)
def export_features_model (line 355)

src/asmsa_v2/featurize/__init__.py
(no local definitions)

src/asmsa_v2/featurize/auto_select.py
def ca_atoms (line 21)
def heavy_atoms (line 36)
def contact_atoms (line 47)
def high_variance_pairs (line 71)
def auto_features (line 92)

src/asmsa_v2/featurize/complex.py
class ComplexMolecule -> __init__ (line 18)
def ComplexMolecule.__init__ (line 51)

src/asmsa_v2/featurize/helpers.py
def extract_dihedral_indices_local (line 9)
def extract_atom_indices (line 93)
def extract_atom_indices._parse (line 95)

src/asmsa_v2/featurize/inter_component.py
class InterComponentDistances -> __init__ (line 17)

def InterComponentDistances.__init__ (line 35)

src/asmsa_v2/featurize/mol_model.py
def _norm_axis_1 (line 18)
class BondsModel -> __init__, forward (line 26)
def BondsModel.__init__ (line 48)
def BondsModel.forward (line 53)
class AnglesModel -> __init__, forward (line 61)
def AnglesModel.__init__ (line 86)
def AnglesModel.forward (line 102)
class DihedralModel -> __init__, forward (line 119)
def DihedralModel.__init__ (line 141)
def DihedralModel.forward (line 146)
class NBDistancesSparse -> __init__ (line 178)
def NBDistancesSparse.__init__ (line 194)
class NBDistancesDense -> __init__ (line 223)
def NBDistancesDense.__init__ (line 236)
class MoleculeModel -> __init__, forward, get_indices (line 254)
def MoleculeModel.__init__ (line 286)
def MoleculeModel.forward (line 310)
def MoleculeModel.get_indices (line 335)

src/asmsa_v2/featurize/molecule.py
def _parse_top (line 14)
def _parse_ff (line 138)
def _match_type (line 197)
class Molecule -> __init__, _match_bonds, _match_angles, _match_dihed, describe_features, get_model, intcoord (line 220)
def Molecule.__init__ (line 252)
def Molecule._match_bonds (line 310)
def Molecule._match_angles (line 343)
def Molecule._match_dihed (line 382)
def Molecule.describe_features (line 443)
def Molecule.get_model (line 458)
def Molecule.intcoord (line 470)

src/asmsa_v2/losses/__init__.py
(no local definitions)

src/asmsa_v2/losses/adversarial.py
def bce_disc_loss (line 10)
def bce_gen_loss (line 22)
def r1_penalty (line 28)

src/asmsa_v2/losses/cv1_bias.py
def cv1_bias_loss (line 6)

src/asmsa_v2/losses/mmd.py
def _imq_kernel (line 21)
def _rbf_kernel (line 27)
def mmd_loss (line 35)

src/asmsa_v2/losses/recon.py
def mse_loss (line 5)
def huber_loss (line 9)

src/asmsa_v2/losses/sliced.py
def sliced_wasserstein_loss (line 19)
def _interp_sorted (line 77)

src/asmsa_v2/md/__init__.py
(no local definitions)

src/asmsa_v2/md/analysis.py
def kT (line 31)
def _parse_fields (line 36)
def read_plumed_file (line 47)
def reweight_weights (line 83)
def reweight_fes (line 151)
def _shift_clip (line 205)
def sum_hills (line 215)
def read_fes_grid (line 279)
def fes_convergence (line 301)
def hill_height_decay (line 325)

src/asmsa_v2/md/gromacs.py
def _flags (line 23)
class Gromacs -> __init__, available, version, _call, script, pdb2gmx, editconf, solvate, grompp, genion, select, make_ndx, mdrun, mdrun_background, trjconv, energy, rms, convert_tpr (line 45)

def Gromacs.__init__ (line 62)
def Gromacs.available (line 79)
def Gromacs.version (line 82)
def Gromacs._call (line 91)
def Gromacs.script (line 102)
def Gromacs.pdb2gmx (line 112)
def Gromacs.editconf (line 118)
def Gromacs.solvate (line 122)
def Gromacs.grompp (line 125)
def Gromacs.genion (line 131)
def Gromacs.select (line 140)
def Gromacs.make_ndx (line 143)
def Gromacs.mdrun (line 147)
def Gromacs.mdrun_background (line 154)
def Gromacs.trjconv (line 185)
def Gromacs.energy (line 196)
def Gromacs.rms (line 199)
def Gromacs.convert_tpr (line 204)
def read_xvg (line 210)
def check_environment (line 217)

src/asmsa_v2/md/mdp.py
def minim_mdp (line 40)
def ions_mdp (line 59)
def nvt_mdp (line 64)
def npt_mdp (line 98)
def production_mdp (line 135)
def ns_to_steps (line 186)
def steps_to_ns (line 198)
def write_mdp_set (line 202)

src/asmsa_v2/md/protocol.py
class ProtocolState -> path, exists (line 25)
def ProtocolState.path (line 31)
def ProtocolState.exists (line 34)
class PreparationProtocol -> __post_init__, _skip, write_mdps, prepare_topology_copy, make_box, solvate, add_ions, minimize, equilibrate_nvt, equilibrate_npt, equilibration_report, production, extend, postprocess_trajectory, status, _repr_html_, __repr__ (line 40)
def PreparationProtocol.__post_init__ (line 76)
def PreparationProtocol._skip (line 85)
def PreparationProtocol.write_mdps (line 93)
def PreparationProtocol.prepare_topology_copy (line 108)
def PreparationProtocol.make_box (line 120)
def PreparationProtocol.solvate (line 128)
def PreparationProtocol.add_ions (line 136)
def PreparationProtocol.minimize (line 146)
def PreparationProtocol.equilibrate_nvt (line 157)
def PreparationProtocol.equilibrate_npt (line 168)
def PreparationProtocol.equilibration_report (line 179)
def PreparationProtocol.production (line 200)
def PreparationProtocol.extend (line 232)
def PreparationProtocol.postprocess_trajectory (line 240)
def PreparationProtocol.status (line 258)
def PreparationProtocol._repr_html_ (line 277)
def PreparationProtocol.__repr__ (line 293)

src/asmsa_v2/md/runner.py
class RunResult -> ok, check, __repr__ (line 22)
def RunResult.ok (line 33)
def RunResult.check (line 36)
def RunResult.__repr__ (line 48)
class Runner -> run, which, available (line 53)
def Runner.run (line 57)
def Runner.which (line 63)
def Runner.available (line 66)
class LocalRunner -> __init__, run (line 70)
def LocalRunner.__init__ (line 85)
def LocalRunner.run (line 91)
class BackgroundJob -> running, returncode, elapsed_s, log, status, wait, kill, __repr__ (line 138)
def BackgroundJob.running (line 147)
def BackgroundJob.returncode (line 151)
def BackgroundJob.elapsed_s (line 155)
def BackgroundJob.log (line 158)
def BackgroundJob.status (line 165)
def BackgroundJob.wait (line 172)
def BackgroundJob.kill (line 175)

def BackgroundJob.__repr__ (line 183)
def run_background (line 187)

src/asmsa_v2/metrics/__init__.py
(no local definitions)

src/asmsa_v2/metrics/kl.py
def _as_2d_np (line 21)
def kl_divergence_knn (line 28)
def prior_match_score (line 88)

src/asmsa_v2/metrics/quality.py
def _as_np (line 33)
def _grade (line 40)
def effective_dimension (line 55)
class QualityReport -> _verdicts, summary, _repr_html_, __repr__ (line 75)
def QualityReport._verdicts (line 89)
def QualityReport.summary (line 141)
def QualityReport._repr_html_ (line 156)
def QualityReport.__repr__ (line 196)
def evaluate_latent (line 200)

src/asmsa_v2/metrics/vamp.py
def _as_np (line 26)
def _sym_inv_sqrt (line 33)
def koopman_singular_values (line 47)
def vamp2_score (line 85)
def implied_timescales (line 98)
def vamp2_scan (line 116)

src/asmsa_v2/models/__init__.py
(no local definitions)

src/asmsa_v2/models/aae.py
class AAE -> __init__, encode, decode, discriminate, forward (line 11)
def AAE.__init__ (line 20)
def AAE.encode (line 47)
def AAE.decode (line 50)
def AAE.discriminate (line 53)
def AAE.forward (line 56)

src/asmsa_v2/models/adapters.py
class PerMolInputAdapter -> __init__, forward (line 10)
def PerMolInputAdapter.__init__ (line 11)
def PerMolInputAdapter.forward (line 18)
class PerMolOutputAdapter -> __init__, forward (line 24)
def PerMolOutputAdapter.__init__ (line 25)
def PerMolOutputAdapter.forward (line 32)

src/asmsa_v2/models/decoder.py
class Decoder -> __init__, forward (line 10)
def Decoder.__init__ (line 11)
def Decoder.forward (line 36)

src/asmsa_v2/models/discriminator.py
class Discriminator -> __init__, forward (line 12)
def Discriminator.__init__ (line 19)
def Discriminator.__init__._lin (line 29)
def Discriminator.forward (line 42)

src/asmsa_v2/models/encoder.py
def _activation (line 9)
class Encoder -> __init__, forward (line 15)
def Encoder.__init__ (line 16)
def Encoder.forward (line 42)

src/asmsa_v2/priors/__init__.py
(no local definitions)

src/asmsa_v2/priors/base.py
class Prior -> __init__, sample, log_prob, to (line 6)
def Prior.__init__ (line 16)
def Prior.sample (line 20)
def Prior.log_prob (line 24)
def Prior.to (line 27)

src/asmsa_v2/priors/gmm.py
class GMMPrior -> __init__, _build, to, sample, log_prob (line 9)
def GMMPrior.__init__ (line 12)
def GMMPrior._build (line 30)

def GMMPrior.to (line 37)
def GMMPrior.sample (line 42)
def GMMPrior.log_prob (line 45)
def gmm_on_circle (line 49)

src/asmsa_v2/priors/image.py
class ImagePrior -> __init__, to, sample, log_prob (line 8)
def ImagePrior.__init__ (line 15)
def ImagePrior.to (line 26)
def ImagePrior.sample (line 32)
def ImagePrior.log_prob (line 42)

src/asmsa_v2/priors/normal.py
class IsotropicNormalPrior -> __init__, _build, to, sample, log_prob (line 7)
def IsotropicNormalPrior.__init__ (line 8)
def IsotropicNormalPrior._build (line 14)
def IsotropicNormalPrior.to (line 19)
def IsotropicNormalPrior.sample (line 24)
def IsotropicNormalPrior.log_prob (line 27)

src/asmsa_v2/train/__init__.py
(no local definitions)

src/asmsa_v2/train/callbacks.py
class Callback -> on_train_begin, on_train_end, on_epoch_begin, on_epoch_metrics, on_epoch_end, on_batch_end (line 9)
def Callback.on_train_begin (line 23)
def Callback.on_train_end (line 24)
def Callback.on_epoch_begin (line 25)
def Callback.on_epoch_metrics (line 26)
def Callback.on_epoch_end (line 27)
def Callback.on_batch_end (line 28)
class CsvLogger -> __init__, on_train_begin, on_epoch_end, on_train_end (line 31)
def CsvLogger.__init__ (line 38)
def CsvLogger.on_train_begin (line 44)
def CsvLogger.on_epoch_end (line 50)
def CsvLogger.on_train_end (line 61)
class EarlyStopping -> __init__, _improved, on_epoch_end (line 67)
def EarlyStopping.__init__ (line 79)
def EarlyStopping._improved (line 98)
def EarlyStopping.on_epoch_end (line 103)
class ModelCheckpoint -> __init__, on_epoch_end, on_train_end (line 144)
def ModelCheckpoint.__init__ (line 151)
def ModelCheckpoint.on_epoch_end (line 169)
def ModelCheckpoint.on_train_end (line 181)
class LiveLossPlot -> __init__, on_epoch_end, on_train_end (line 186)
def LiveLossPlot.__init__ (line 195)
def LiveLossPlot.on_epoch_end (line 217)
def LiveLossPlot.on_train_end (line 270)
class LatentSnapshot -> __init__, on_epoch_end, plot (line 277)
def LatentSnapshot.__init__ (line 286)
def LatentSnapshot.on_epoch_end (line 294)
def LatentSnapshot.plot (line 300)

src/asmsa_v2/train/trainer.py
class Trainer -> __init__, _latent_penalty, _disc_step, _set_disc_requires_grad, _recon_target, _unpack, _train_one_batch, _clip, evaluate, _warn_if_val_loader_shuffles, fit, _apply_warmup, _track_best, _warn_batch_size_mismatch, encode_all, reconstruct_all, save_checkpoint, load_checkpoint (line 50)
def Trainer.__init__ (line 62)
def Trainer._latent_penalty (line 127)
def Trainer._disc_step (line 139)
def Trainer._set_disc_requires_grad (line 163)
def Trainer._recon_target (line 168)
def Trainer._unpack (line 180)
def Trainer._train_one_batch (line 186)
def Trainer._clip (line 253)
def Trainer.evaluate (line 261)
def Trainer._warn_if_val_loader_shuffles (line 302)
def Trainer.fit (line 317)
def Trainer._apply_warmup (line 408)
def Trainer._track_best (line 420)
def Trainer._warn_batch_size_mismatch (line 439)
def Trainer.encode_all (line 453)
def Trainer.reconstruct_all (line 481)
def Trainer.save_checkpoint (line 505)
def Trainer.load_checkpoint (line 526)

src/asmsa_v2/tune/__init__.py
(no local definitions)

src/asmsa_v2/tune/analyzer.py
class TuningAnalyzer -> __init__, best_params, table, print_best, plot_scores, plot_curves, plot_param_effect, plot_overview, _repr_html_, __repr__ (line 18)
def TuningAnalyzer.__init__ (line 28)
def TuningAnalyzer.best_params (line 41)
def TuningAnalyzer.table (line 44)
def TuningAnalyzer.print_best (line 49)
def TuningAnalyzer.plot_scores (line 60)
def TuningAnalyzer.plot_curves (line 93)
def TuningAnalyzer.plot_param_effect (line 123)
def TuningAnalyzer.plot_overview (line 155)
def TuningAnalyzer._repr_html_ (line 194)
def TuningAnalyzer.__repr__ (line 197)
def _on (line 201)

src/asmsa_v2/tune/runner.py
def run_config (line 17)
def main (line 64)

src/asmsa_v2/tune/space.py
class SearchSpace -> to_dict, suggest, sample_random, size (line 15)
def SearchSpace.to_dict (line 50)
def SearchSpace.suggest (line 55)
def SearchSpace.sample_random (line 71)
def SearchSpace.size (line 91)
def params_to_configs (line 132)

src/asmsa_v2/tune/tuner.py
class KLTracker -> __init__, on_epoch_metrics, score (line 42)
def KLTracker.__init__ (line 50)
def KLTracker.on_epoch_metrics (line 54)
def KLTracker.score (line 66)
class Trial -> to_dict, from_dict (line 78)
def Trial.to_dict (line 90)
def Trial.from_dict (line 99)
class TuningResult -> __post_init__, best, best_params, top, to_configs, to_dataframe, save, load, summary, _repr_html_, __repr__ (line 104)
def TuningResult.__post_init__ (line 113)
def TuningResult.best (line 122)
def TuningResult.best_params (line 128)
def TuningResult.top (line 131)
def TuningResult.to_configs (line 134)
def TuningResult.to_dataframe (line 141)
def TuningResult.save (line 154)
def TuningResult.load (line 165)
def TuningResult.summary (line 173)
def TuningResult._repr_html_ (line 190)
def TuningResult.__repr__ (line 222)
def _score_trial (line 226)
def tune (line 266)
def tune.run_one (line 328)
class _PruneReporter -> __init__, on_epoch_end (line 439)
def _PruneReporter.__init__ (line 442)
def _PruneReporter.on_epoch_end (line 447)

src/asmsa_v2/utils/__init__.py
(no local definitions)

src/asmsa_v2/utils/config.py
class ModelConfig -> to_dict, from_dict, from_seed (line 8)
def ModelConfig.to_dict (line 19)
def ModelConfig.from_dict (line 24)
def ModelConfig.from_seed (line 29)
class TrainConfig -> __post_init__, uses_discriminator, to_dict, from_dict (line 44)
def TrainConfig.__post_init__ (line 95)
def TrainConfig.uses_discriminator (line 119)
def TrainConfig.to_dict (line 122)
def TrainConfig.from_dict (line 127)
def resolve_device (line 135)

src/asmsa_v2/utils/project.py
class Project -> __post_init__, dir, path, ensure_dirs, conf_path, traj_path, _resolve, features_file, scaler_file, split_file, molecule_file, tuning_file, checkpoint_file, cv_stem, plumed_model_file, batch_model_file, plumed_dat, to_dict, save, load, stage_status, check, summary, _repr_html_, __repr__, make_prior (line 42)

def Project.__post_init__ (line 112)
def Project.dir (line 129)
def Project.path (line 137)
def Project.ensure_dirs (line 141)
def Project.conf_path (line 146)
def Project.traj_path (line 150)
def Project._resolve (line 153)
def Project.features_file (line 161)
def Project.scaler_file (line 163)
def Project.split_file (line 165)
def Project.molecule_file (line 167)
def Project.tuning_file (line 169)
def Project.checkpoint_file (line 171)
def Project.cv_stem (line 174)
def Project.plumed_model_file (line 176)
def Project.batch_model_file (line 179)
def Project.plumed_dat (line 182)
def Project.to_dict (line 185)
def Project.save (line 191)
def Project.load (line 206)
def Project.stage_status (line 237)
def Project.check (line 251)
def Project.check.add (line 260)
def Project.summary (line 333)
def Project._repr_html_ (line 349)
def Project.__repr__ (line 379)
def Project.make_prior (line 383)
class _UniformPrior -> __init__, to, sample, log_prob (line 410)
def _UniformPrior.__init__ (line 413)
def _UniformPrior.to (line 417)
def _UniformPrior.sample (line 421)
def _UniformPrior.log_prob (line 425)
class PreflightReport -> failures, ok, raise_if_failed, summary, _repr_html_, __repr__ (line 435)
def PreflightReport.failures (line 442)
def PreflightReport.ok (line 446)
def PreflightReport.raise_if_failed (line 449)
def PreflightReport.summary (line 455)
def PreflightReport._repr_html_ (line 465)
def PreflightReport.__repr__ (line 486)
def _have (line 490)

src/asmsa_v2/utils/seeding.py
def set_seed (line 8)

src/asmsa_v2/viz/__init__.py
(no local definitions)

src/asmsa_v2/viz/diagnostics.py
def _np (line 15)
def feature_scale_report (line 21)
def training_curves (line 68)
def latent_overview (line 101)
def vamp_scan_plot (line 154)
def reconstruction_report (line 206)
def fes_plot (line 237)
def convergence_plot (line 256)

src/asmsa_v2/viz/interactive.py
def _to_np (line 20)
def _require_plotly (line 34)
def interactive_latent (line 42)
def latent_browser (line 100)
def latent_browser._render (line 137)
def latent_browser._on_click (line 154)

src/asmsa_v2/viz/latent.py
def _to_np (line 13)
def scatter_latents (line 19)
def hist2d_latent (line 40)
def kde_latent (line 50)
def free_energy_surface (line 65)
def latent_colored_by (line 127)
def latent_trajectory (line 153)

src/asmsa_v2/viz/properties.py
def radius_of_gyration (line 14)
def rmsd_to (line 19)

`def component_distance` (line 26)
`def contact_count` (line 40)
`def ligand_rmsd_after_protein_alignment` (line 54)
`def standard_properties` (line 74)

`src/asmsa_v2/viz/structure.py`
`def _require_p3d` (line 20)
`def _frame_pdb` (line 28)
`def view_frame` (line 35)
`def view_frames_grid` (line 91)

`src/asmsa_v2/viz/style.py`
`def use_asmsa_style` (line 37)
`def latent_axes` (line 44)

Appendix B - reading the inventory

The version-specific source-to-source correspondence and behavior changes are discussed in Part III. Files retained from the old package may preserve some feature calculations exactly, while surrounding API, framework, training loop, and export behavior differ. Lines in this index are locations in the checked-out source snapshot and may change in later commits.